μLM: Rethinking Sub-100M Language Models through Memory-First Design

Anonymous Author(s)
Affiliation
Address
email

Abstract

Ubiquitous IoT devices, such as companion toys, wearables, and micro-robots, are increasingly embracing language models, yet current cloud-based deployments suffer from latency, privacy, and weak-network reliability issues. These resource-constrained devices typically cap deployable model size below 100M parameters, while existing sub-100M models fall short in basic language and commonsense capabilities. In this extreme regime, we argue that the memory bottleneck supersedes the traditional compute bottleneck as the binding constraint, calling for memory-first design that maximizes model capability under a fixed parameter budget. Notably, the stringent budget binds both what the model stores and what it can afford to learn, demanding architecture-data co-design. In this paper, we propose μLM, a memory-first sub-100M language model framework instantiating this co-design. For architecture, we present DMLA, which preserves latent attention's compact cache while relieving its feature-squeezing bottleneck under aggressive compression via explicit content-position latent decoupling. For data, we present μMix, a capacity-aware multi-stage curriculum tailored to sub-100M models, prioritizing high-quality natural language and commonsense data aligned with embedded scenarios over brute-force token scaling. μLM-70M surpasses all sub-300M baselines and rivals several sub-1B models on standard benchmarks; even at 9M, it remains competitive. Fine-tuned into a tool-calling agent, μLM outperforms same-scale baselines and achieves a 97.2% end-to-end executable rate, presenting a promising direction toward practical sub-100M language modeling.

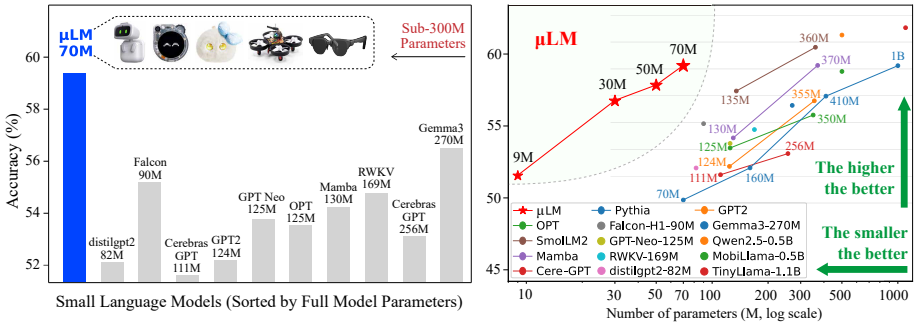


Figure 1: **μLM Performance Comparison. Left:** Average accuracy on standard zero-shot benchmarks across sub-300M SLMs. The icons illustrate the constrained devices that motivate our memory-first design. **Right:** Accuracy-parameter trade-off. The μLM family (red stars) establishes a new performance frontier in the sub-100M regime, evidencing that capability density at extreme scale benefits from memory-first architecture-data co-design. Full results are provided in Table 1.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

1 Introduction

The capability density of small language models (SLMs) has been improving rapidly, driven by advances including model architecture, data recipes, and training strategies [1]. Recent Gemma- and MiniCPM-family models [2, 3] demonstrate remarkable parameter efficiency, accelerating the deployment of generative AI on edge devices. However, most SLMs are designed primarily for phone- or laptop-class platforms, while a broader and more pervasive class of ultra-constrained devices remains underserved, including companion toys, voice-recording cards, wearables, and micro-robots. For these devices, cloud-only inference is not a satisfactory default: inputs are private and continuously generated, interactions are latency-sensitive, and connectivity is often intermittent or costly to sustain [4, 5]. It remains reasonable for open-ended or heavy queries, but basic language understanding and intent routing for these devices should remain local and always available.

Unlike phone-class devices, ultra-constrained platforms (e.g., RP3A0 [6], STM32N6 [7], CV1800B [8]) operate under system memory ranging from only tens to a few hundred megabytes. Crucially, the operating system, sensor buffers, I/O stacks, and application logic share the same memory budget alongside model weights and runtime states, pushing deployable backbones toward the sub-100M regime [9, 10]. Yet language models at this scale currently fall short of practical interaction needs [11–14]. This extreme scenario reveals a design paradigm that differs fundamentally from mainstream on-device LLMs: *the memory bottleneck supersedes the compute bottleneck as the primary limiting factor.* A model that exceeds the available memory cannot be accelerated [15], while a model that fits only through naive shrinking is often too weak to be useful. The central challenge is thus to design a memory-first language model that maximizes capability density under strict limitations.

This constraint exposes two complementary design axes. On the architecture side, reducing KV state alone is not enough: aggressive KV-head sharing such as MQA or small-group GQA reduces per-head KV diversity, while standard latent attention improves the trade-off yet may entangle semantic matching and positional information under aggressive sub-100M compression [16–18]. On the data side, once the memory budget fixes the model below 100M parameters, pretraining becomes a capacity-allocation problem. Tiny models cannot absorb all domains simultaneously, and brute-force token scaling or high-format-cost corpora consume scarce capacity without improving the target interaction abilities. Therefore, sub-100M language modeling requires not only a memory-efficient architecture, but also data schedules that prioritize what the model can afford to learn.

In this paper, we propose μLM, a sub-100M language model framework built on architecture-data co-design. On the architecture side, we adopt latent attention for its superior memory–expressivity trade-off under a fixed parameter budget [18], and further propose DMLA (Decoupled Multi-head Latent Attention), an attention mechanism reorganized around memory-first deployment. At its core, DMLA explicitly factorizes the latent cache into content and position sub-spaces, alleviating feature squeezing under aggressive compression through input-aware dual-stream fusion. DMLA further introduces ALiBiLe, a learnable positional prior naturally compatible with latent caching. On the data side, we propose μMix, a data scheduling recipe tailored to sub-100M capacity constraints. Instead of scaling token volume alone [19, 20], μMix couples a capacity-aware data mixture with a multi-stage curriculum that progressively establishes foundational syntax and fluency, consolidates the language–logic backbone, and finally injects commonsense knowledge.

Experimental results show that μLM-70M surpasses all sub-300M baselines on standard zero-shot benchmarks [21], and rivals several sub-1B models (Figure 1). The μLM family establishes a new performance frontier in the sub-100M regime. To further demonstrate practical value, we fine-tune μLM into μAgent, a variant designed for tool-calling [22] that achieves a 97.2% end-to-end executable rate on out-of-distribution scenarios, outperforming its same-scale baseline.

To summarize, our work makes the following key contributions:

- *Memory-first design framework.* We articulate a memory-first design framework for sub-100M language models, identify deployment-memory footprint rather than raw compute as the binding bottleneck, and derive two design principles governing KV allocation and positional injection.
- *Architecture-data co-design.* We propose μLM, jointly addressing the architectural and data axes of sub-100M design: **DMLA** instantiates the memory-first principles by decoupling the latent cache into content and position routes with input-aware fusion to alleviate feature squeezing, while μ**Mix** prioritizes learning efficiency over token volume through a capacity-aware multi-stage curriculum.

- *Comprehensive validation.* We evaluate μLM across standard zero-shot benchmarks and lightweight agentic tasks: μLM-70M surpasses all sub-300M baselines and rivals several sub-1B models, while the fine-tuned μAgent outperforms its same-scale baseline and achieves a 97.2% executable rate.

2 Related Work

On-device generative AI. A broad class of battery-powered edge devices, including companion toys, wearables, and micro-robots, increasingly relies on generative intelligence, yet current cloud-based deployments suffer from latency, privacy, and offline-availability limitations, creating a pressing need for on-device inference [5, 23, 24]. Unlike smartphones, these devices operate under milliwatt-scale power budgets and megabyte-scale memory [9, 25], which is further shared by the operating system, sensor buffers, and application logic. Thus, the practical memory available to models is ultra-constrained, often pushing deployable backbones below 100M parameters [4, 26]. However, mainstream SLMs such as MobileLLM [27], MobiLlama [28], and SmolLM2 [20] mainly target the 100M–1B range [29], while existing sub-100M models fall short of practical interaction quality [11, 13, 14]. To our knowledge, μLM is the first language model framework explicitly designed for this regime, surpassing several hundred-million-parameter models through a memory-first design.

Memory-efficient architectures and data-centric pretraining. For architecture, efficient designs reduce per-head key/value diversity (MQA [16], GQA [17]) or compress the cache into a shared latent space (MLA [18, 30]), while positional encodings such as RoPE [31] and ALiBi [32] interact non-trivially with these mechanisms, a cache-compatibility issue we revisit in Section 3. For data, scaling laws and data-constrained studies [19, 33], together with curated corpora such as FineWeb, DataComp-LM, and Cosmopedia [34–36], highlight the importance of token budgets, mixture design, and synthetic data [37]. However, neither line has systematically examined the appropriate attention and data choices under sub-100M constraints, where every parameter must be justified against an extreme capacity ceiling. μLM, to our knowledge, is the first to co-design architecture and data under an explicit memory-first objective for this regime: DMLA alleviates feature squeezing via content-position decoupling, while μMix allocates limited capacity using a capacity-aware curriculum.

3 Memory-First Design Principles

Two Design Questions. On ultra-constrained devices, deployment feasibility is determined by total memory footprint before throughput optimization becomes meaningful. Design should therefore preferentially compress what persists during inference, model weights and cached states, rather than minimize transient compute. Two questions follow that we address in turn: (i) how should scarce KV parameters be allocated, and (ii) how should positional information be injected under that allocation?

(i) Latent attention over grouped sharing. Under a fixed total parameter budget, Grouped-Query Attention (GQA) [17] and Multi-head Latent Attention (MLA) [18] allocate KV parameters in qualitatively different ways: GQA spends its budget on g shared key/value projections, each used by a group of heads, while MLA spends it on a shared low-rank backbone followed by per-head up-projections. With hidden size D, h query heads of width $d = D/h$, and matched non-KV components, equating KV parameter counts yields $r = \frac{2gd}{3}$, where r is the MLA latent width. Which allocation is better depends on the structure of the target KV mapping. Prior empirical studies show that learned KV representations exhibit low intrinsic dimensionality and compressible spectral structure [38–40], so the shared backbone efficiently encodes the global KV geometry; what limits expressivity is the number of distinct per-head projections. We formalize this as Theorem 1 in Appendix B.1: when a target mapping factorizes through a rank-r latent but requires $m > g$ distinct per-head projections, equal-budget MLA represents it exactly while GQA cannot. Under tight parameter budgets that force g to be small, particularly the extreme $g = 1$ (MQA) case [16], GQA's head-diversity ceiling becomes the binding constraint, motivating MLA as the starting point for sub-100M models.

(ii) ALiBi-style bias over RoPE. We argue that ALiBi-style biases [32] are architecturally preferable to RoPE [31] once latent attention is adopted, for two reasons. First, with limited capacity, an explicit locality prior is more efficient than asking the model to discover one. RoPE encodes relative position through query-key rotations, a flexible mechanism that does not impose distance

decay, leaving the model to learn it from data alongside semantics, syntax, and task behavior. ALiBi instead injects an additive logit bias that decays linearly with distance, placing the recency prior directly in the attention score, and freeing capacity for content modeling. Second, RoPE imposes a compatibility-driven positional split, not a modeling-driven one. The query-key rotation cannot be factored out of the cached state without re-computing it on every query, forcing RoPE-equipped MLA to reserve a fraction of each head's width purely as inert rotational carriers (Appendix B.3). ALiBi entering as a post-dot-product bias imposes no such reservation: any subsequent partition of head width then becomes a deliberate modeling choice rather than a structural tax. We exploit precisely this freedom in Section 4.2, where DMLA's per-head split into content and position streams is allocated to two independently-routed attention pathways, not to passive position-carrying.

4 μLM Architecture

4.1 Overview

Section 3 identifies two architectural ingredients best suited to sub-100M memory-first deployment: *latent KV compression* to minimize the persistent decoding state, and an *ALiBi-style positional prior* to avoid the mandatory positional-carrier subspace imposed by RoPE. The remaining design question is how to organize the compressed state so that a tiny model can better exploit its limited capacity.

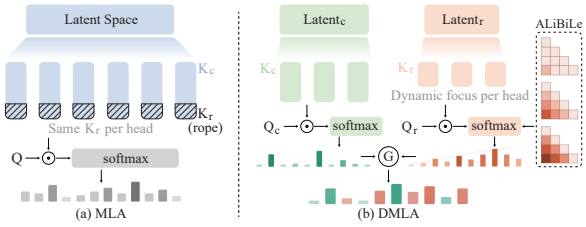


Figure 2: Comparison with MLA and proposed DMLA.

We propose **DMLA** (**D**ecoupled **M**ulti-head **L**atent **A**ttention), which restructures the latent cache into two specialized routes, a content route and a position-biased route, compressed, routed, and normalized independently, fusing only after each forms its own attention distribution. Unlike RoPE-based MLA, where content and positional channels are separated primarily for compatibility with compressed caching, DMLA elevates this separation from a compatibility constraint to a deliberate modeling choice. Figure 2 contrasts the two mechanisms.

4.2 Active Latent Factorization

Let $X \in \mathbb{R}^{T\times D}$ denote hidden states, h the head count, and m_c, m_r the content and position latent widths. DMLA first builds two independent latent caches:

$$C^{\ell} = \mathrm{Norm}(XW_{\ell}^{D}), \quad \ell \in \{c, r\}. \tag{1}$$

The persistent memory footprint per layer per token is bounded to $m_c + m_r$ scalars; keys and values are reconstructed on the fly from this state, while queries are computed from the current hidden state.

From compatibility-driven separation to active factorization. In RoPE-compatible MLA, content and positional score components may be algebraically separable, but they are summed and jointly normalized by a single softmax (Fig. 2a). DMLA makes a stronger choice: the two routes form independent attention distributions before fusion. Defining $\mathcal{A}_d(Q, K, V; B) = \mathrm{softmax}(QK^{\top}/\sqrt{d} + B)\,V$, DMLA computes:

$$Y^c = \mathcal{A}_{d_c}(Q^c, K^c, V;\, M), \qquad Y^r = \mathcal{A}_{d_r}(Q^r, K^r, V;\, M + A^{\mathrm{ALiBiLe}}), \tag{2}$$

where M is the causal mask. The content route produces position-bias-free semantic matching; the position-biased route additionally incorporates **ALiBiLe** (**ALiBi** with **Le**arnable slopes), which retains ALiBi's linear distance-bias form but makes per-head slopes ρ_u trainable, with $A^{\mathrm{ALiBiLe}}_{u,t,s} = -\rho_u(t-s)$ for $s \le t$, where u indexes attention heads and t, s are query/key positions (Appendix D.3). The original ALiBi schedule was designed for models with tens of heads, where a suboptimal per-head slope can be compensated by others; in our regime ($h = 8$–16), every head's effective receptive field carries disproportionate weight, and a fixed geometric schedule risks wasting limited routing capacity.

Crucially, the position-biased route is *not* a fixed distance kernel: the query-key interaction $Q^r(K^r)^{\top}$ further modulates the ALiBiLe prior, allowing content-aware selection within the recency-biased

window, consistent with evidence that positional attention in Transformers is inherently content-dependent [41, 42]. The route therefore acts as a *local expert*: its attention is shaped by the positional prior but not dictated by it. Because softmax is non-linear, the two routes' interaction moves from the score level to the distribution level, where each forms its own routing hypothesis before being combined, expanding the representable family of attention patterns beyond what a single jointly-normalized softmax can express.

Why factorization helps tiny models. A single highly compressed latent in a sub-100M model is an extreme information bottleneck: forcing it to simultaneously encode semantic alignment, recency bias, and head-specific routing leads to severe *feature squeezing*. DMLA provides explicit structural relief by allocating two separate bottlenecks, encouraging the model to compress content and positional evidence differently before attention is formed. The split also improves parameter efficiency by eliminating cross-route key up-projections (Appendix B.2).

4.3 Complementary Design Choices

The active factorization above defines DMLA's core. We complete the data path (Fig. 3) with three coordinated choices.

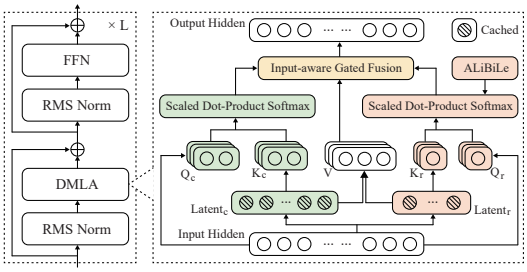


Figure 3: Architecture of μLM.

Asymmetric stream construction. Queries are generated full-rank from the hidden state and split into the two streams $(Q^c, Q^r) = \mathrm{split}_h(XW^Q)$, since they are ephemeral and compressing them would constrain routing without saving deployment memory. Keys are read exclusively from their corresponding latent, $K^\ell = \mathrm{split}_h(C^\ell W_\ell^K)$, preserving route specialization, and remain multi-head rather than shared, since collapsing all heads into a single positional prototype would undermine factorization. Values, however, are computed once from the joint latent $V = \mathrm{split}_h([C^c; C^r]\, W^V)$. This is justified on two grounds. *Modeling:* since V does not participate in the $QK^\top$ score, it carries no content/positional role conflict and benefits from reading the full information in both latents. *Computation:* because the two routes' attention probabilities P^c, P^r share the same shape and are linearly combined by the stream gate (Eq. 3), the bilinearity of attention permits fusing first and multiplying once: $Y_{\text{fused}} = (G \odot P^c + (1 - G) \odot P^r)\, V$, rather than computing P^cV and P^rV separately.

Post-softmax stream gate. To adaptively fuse the two routes, DMLA introduces an input-conditioned gate:

$$G = \sigma\big(g_0 + \alpha\, \tanh(XW_g)\big), \qquad Y_{\text{fused}} = G \odot Y^c + (1 - G) \odot Y^r, \tag{3}$$

where $g_0 \in \mathbb{R}^h$ is a learned per-head static prior, $W_g \in \mathbb{R}^{D\times h}$, and α scales the input-aware modulation. The gate acts *after* the two independent softmaxes, selecting between two already-formed routing hypotheses rather than reweighting score logits. Because it interpolates between normalized outputs, fusion is inherently scale-stable; input-conditioning further allows each token and each head to dynamically shift between semantic matching and recency bias (Appendix D.4).

Overall architecture. Beyond DMLA (Fig. 3, right), μLM follows a standard pre-norm Transformer [43]: RMSNorm [44] before each sub-layer, dense SwishGLU [45] feed-forward (we deliberately avoid MoE, whose routing tables inflate the static weight footprint), and tied input-output embeddings with a 16K-token vocabulary chosen to keep the embedding table under 20% of total parameters at sub-100M scale. The complete block is $\mathrm{DMLA}(X) = \mathrm{merge}_h(Y_{\text{fused}})\, W^O$.

Two properties make DMLA Pareto-improving over MLA at matched latent budget. First, it introduces *no additional cached state* beyond the $m_c + m_r$ latent in Eq. 1, the stream gate and all Q/K/V projections are computed on the fly. Second, it adds *no additional QK dot-product compute*: MLA's single $Q(K)^\top$ with per-head width $d_c{+}d_r$ splits into two narrower products $Q^c(K^c)^\top$ and $Q^r(K^r)^\top$, yielding the same $hT^2(d_c + d_r)$ operations. The stream gate adds only $Dh + h$ parameters, under 0.02% of the 70M-scale total.

5 μMix: Capacity-Aware Data Recipe and Curriculum

For sub-100M models, data mixture directly determines what the model can learn: parameter capacity is too limited to absorb all domains simultaneously, so every corpus-level decision is strictly a capacity-allocation decision. Changing the mixture does not merely affect convergence speed; it dictates what the model can afford to remember and generalize. This section describes μMix, a data scheduling recipe organized around three capacity-aware principles.

5.1 Three Capacity-Aware Principles

P1: Total scale balanced against the capacity wall. The Chinchilla compute-optimal ratio of $\sim$20 tokens/parameter [19] is far too low for sub-100M models, yet feeding hundreds of billions of loosely filtered tokens hits a capacity wall, the point at which held-out loss on a high-quality reference set plateaus ($< 0.5\%$ improvement per additional 1B tokens) and further data mostly introduces conflicting long-tail facts that cannot be reliably stored [33, 46]. μMix uses an approximately 10B-token physical corpus ($\sim$140 tokens/param at 70M), located near the empirically observed plateau in this regime, and trains for 5 epochs to amplify within-budget signal.

P2: Aggressive omission of high-format-cost domains. μMix excludes standalone code and math corpora from base pre-training. Code (indentation, brackets, long-range formal dependencies) and dense mathematical notation impose a disproportionate format cost on sub-100M budgets, consuming capacity for surface regularities that do not transfer to general edge deployment. We allocate capacity exclusively to clean natural language, educational explanations, and everyday commonsense, following the textbook-quality logic of the Phi series [37, 47].

P3: Macro-curriculum over one-shot mixing. Micro-models exhibit an exacerbated recency bias: swapping the mixture in the final 10% of training shifts HellaSwag and PIQA by 1.5-2.8 points at 50M. Rather than treating this as a vulnerability, μMix exploits it as a control mechanism by coupling the data schedule with a WSD-style (Warmup–Stable–Decay) learning-rate schedule [48, 49]: the LR decay phase coincides with a mixture shift toward curated high-quality sources, gently steering the already-formed language–logic backbone rather than forcing it to relearn the world.

5.2 The μMix Recipe

Sources. μMix draws on five corpora (Fig. 4): FineWeb-Edu [34] (educational web text), Cosmopedia [36] (synthetic textbook-style explanations), TinyStories [14] (highly grammatical narratives), WikiHow [50] (procedural commonsense), and Simple English Wikipedia (compact factual grounding without the entity density of full encyclopedias).

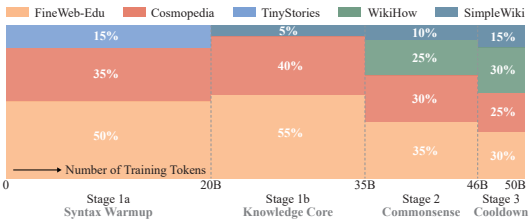


Figure 4: Curriculum learning schedule of μMix.

Stages 1a-1b: Linguistic foundation. Stage 1a (Syntax Warmup) pairs TinyStories with educational text to give the model a clean early signal for grammar and short-range coherence. TinyStories is then phased out in **Stage 1b (Knowledge Core)** to prevent over-specialization to a child-like narrative style, a regression we directly observed when stories were retained throughout, and a small share of Simple English Wikipedia is introduced early to begin acclimating the model to encyclopedic register before commonsense data arrives in bulk.

Stage 2 (Commonsense Injection). The bulk of WikiHow and Simple Wikipedia enters here. We retain Cosmopedia and FineWeb-Edu as a stable trunk to prevent abrupt distribution shifts; the new corpora supply procedural and factual commonsense without displacing the language-logic backbone built in Stage 1.

Stage 3 (Cooldown). Stage 3 uses a small physical budget with higher replay weight (5 epochs, following [33]) and is paired with LR decay. The mixture continues the trajectory established in Stage 2 by further increasing the share of WikiHow and Simple English Wikipedia while modestly reducing FineWeb-Edu and Cosmopedia. The intent is not to introduce new data sources but to consolidate

Table 1: Performance comparison between different SLMs.

Model	#Params	Attention Mechanism	Position Encoding	Language	Contextual Understanding			Commonsense		Score(%)↑ (Averaged)
				BLiMP↑	BoolQ↑	WinoGrande↑	SIQA↑	PIQA↑	HellaSwag↑	
number of parameters < 300M										
Pythia-70M	70M	MHA	RoPE	0.7326	0.5196	0.5185	0.3526	0.5909	0.2751	49.82
distilgpt2-82M	82M	MHA	Learned Abs.	0.7873	0.5884	0.5036	0.3608	0.6099	0.2750	52.08
Falcon-H1-Tiny-90M	90M	GQA+SSM	RoPE	0.8019	0.5869	0.5107	0.3823	0.6518	0.3762	55.16
Cerebras-GPT-111M	111M	MHA	Learned Abs.	0.7649	0.6214	0.4901	0.3552	0.5941	0.2710	51.61
GPT2-124M	124M	MHA	Learned Abs.	0.8215	0.4872	0.5162	0.3659	0.6289	0.3114	52.19
GPT-Neo-125M	125M	MHA	Learned Abs.	0.7974	0.6171	0.5043	0.3721	0.6306	0.3040	53.76
OPT-125M	125M	MHA	Learned Abs.	0.8176	0.5618	0.5130	0.3715	0.6284	0.3158	53.47
Mamba-130M	130M	SSM	Implicit	0.8043	0.5523	0.5217	0.3756	0.6447	0.3526	54.19
SmolLM2-135M	135M	GQA	RoPE	0.8010	0.6046	0.5304	0.3956	0.6823	0.4318	57.43
Pythia-160M	162M	MHA	RoPE	0.7744	0.5511	0.5114	0.3654	0.6181	0.3019	52.04
RWKV-169M	169M	RWKV	Implicit	0.8242	0.6043	0.5178	0.3695	0.6474	0.3218	54.75
Cerebras-GPT-256M	256M	MHA	Learned Abs.	0.7892	0.6150	0.5130	0.3680	0.6137	0.2860	53.08
Gemma3-270M	270M	MQA	RoPE	0.8008	0.5798	0.5200	0.4007	0.6757	0.4090	56.43
number of parameters > 300M										
OPT-350M	331M	MHA	Learned Abs.	0.8267	0.5777	0.5233	0.3987	0.6507	0.3684	55.76
GPT2-355M	355M	MHA	Learned Abs.	0.8266	0.5859	0.5312	0.3910	0.6763	0.3938	56.75
SmolLM2-360M	362M	GQA	RoPE	0.8161	0.6202	0.5250	0.4053	0.7165	0.5450	60.47
Mamba-370M	371M	SSM	Implicit	0.8195	0.6214	0.5533	0.4002	0.6937	0.4649	59.22
Pythia-410M	405M	MHA	RoPE	0.8162	0.6073	0.5391	0.3900	0.6687	0.4063	57.13
Qwen2.5-0.5B	494M	GQA	RoPE	0.8254	0.6205	0.5643	0.4432	0.7035	0.5224	61.32
MobiLlama-0.5B	500M	MHA	RoPE	0.7919	0.5590	0.5288	0.4022	0.7203	0.5252	58.79
Pythia-1B	1011M	MHA	RoPE	0.8320	0.6086	0.5328	0.4017	0.7051	0.4713	59.19
TinyLlama-1.1B	1100M	GQA	RoPE	0.8082	0.5804	0.5880	0.4115	0.7296	0.5922	61.83
μLM-70M	74M	DMLA	ALiBiLe	0.7819	0.6367	0.5120	0.4339	0.6886	0.5018	59.25
μLM-50M	55M	DMLA	ALiBiLe	0.7714	0.6087	0.5177	0.3858	0.6779	0.4916	57.55
μLM-30M	37M	DMLA	ALiBiLe	0.7653	0.5968	0.5325	0.3796	0.6682	0.4661	56.81
μLM-9M	9M	DMLA	ALiBiLe	0.6439	0.5836	0.5210	0.3610	0.6533	0.3111	51.23

procedural and factual commonsense as the model's last impression, exploiting the recency sensitivity quantified in P3 above to bias final behavior toward grounded everyday knowledge rather than toward generic web text.

In summary, μMix navigates the capacity wall of sub-100M models by aggressively omitting high-format-cost domains, physically separating linguistic foundation from commonsense injection, and exploiting micro-model recency sensitivity via WSD-coupled annealing.

6 Experiments

6.1 Main Results

Evaluation setup. We evaluate base language model capability using `lm-evaluation-harness` [21] under a zero-shot setting without prompt tuning. We use six standard benchmarks covering the three capability clusters most relevant to on-device interaction: linguistic knowledge (BLiMP [51]), contextual understanding (BoolQ [52], WinoGrande [53], SIQA [54]), and commonsense reasoning (PIQA [55], HellaSwag [56]). The reported score is the unweighted average of the six task accuracies, multiplied by 100. All baselines are obtained from publicly available checkpoints [1] listed with their Hugging Face identifiers in Appendix E.

Results. Table 1 compares μLM with representative SLMs ranging from sub-100M models to 1B-class baselines. In the sub-300M regime, μLM-70M achieves the best averaged performance despite using substantially fewer parameters. More importantly, μLM-50M reaches 57.55, on par with the strongest sub-300M baselines while using less than half the parameters. The 30M-class model remains highly competitive, surpassing several models with 100M–300M parameters, while even μLM-9M achieves 51.23, above Pythia-70M and close to multiple 100M-class baselines. The comparison with larger models further highlights the capability density of μLM. μLM-70M matches or exceeds several 300M–1B baselines, although it remains smaller by more than an order of magnitude in some cases. We note that μLM is intentionally limited to a maximum context length of 1024 tokens. Ultra-constrained IoT devices are typically dominated by command- or utterance-level interaction, where inputs are short, user intent is explicit, and responses must be produced with low latency [57, 58]. We therefore did not extend context length to chase benchmark scores.

[1] MobileLLM [27] is not included in our evaluation as its checkpoints are gated and not publicly accessible.

Table 2: Ablations on (a) attention mechanism, (b) DMLA components, and (c) curriculum stages.

Method	KV	Layers	Hidden	Params	Score
MHA	12	9	768	76.30M	56.27
MQA	1	10	768	72.56M	55.84
GQA	4	10	768	75.51M	56.38
MLA	12	10	768	76.13M	56.50
DMLA	**12**	**10**	**768**	**74.13M**	**57.62**

(a) Attention comparison (∼70M).

ALiBiLe	Decoupled	Fused_V	Gated	Score
				56.50
√				56.64
√	√			56.86
√	√	√		57.25
√	√	√	√	**57.62**

(b) DMLA components.

Curriculum	Score
Cosine baseline	57.62
Single-stage	58.34
Two-stage	58.91
Three-stage	**59.25**
Four-stage	59.18

(c) Curriculum stages.

6.2 Ablation Study

Evaluation setup. Table 2 validates the two main ingredients of μLM: DMLA on the architecture side and μMix on the data side. Unless otherwise specified, all ablations are conducted at the 70M scale and use the same tokenizer, optimizer, training-token budget, and six-task evaluation suite as in the main experiment. The attention-mechanism and DMLA-component ablations are trained with the same cosine-decay recipe and matched data exposure.

Attention mechanism. Table 2(a) compares MHA, MQA, GQA, MLA, and DMLA under comparable model sizes. MQA reduces KV state but performs worst, indicating that overly aggressive sharing hurts expressivity at this scale. GQA improves over MQA and MHA, but remains below MLA, confirming the value of latent KV compression for memory-first models. However, the MLA baseline still reaches only 56.50. DMLA improves this to 57.62, gaining 1.12 points over MLA while using fewer parameters. This suggests that the bottleneck in tiny latent-attention models is not merely cache size, but the organization of the compressed state.

DMLA components. Table 2(b) further decomposes DMLA. Starting from the MLA-style latent baseline, adding ALiBiLe gives a small gain, showing that learnable distance priors are helpful but insufficient by themselves. Decoupling the latent routes brings an additional gain, but the improvement is moderate because using the content latent to form values, as in the MLA default, is already a reasonable design. The larger gains appear when decoupling is paired with FUSED-V and gated fusion. FUSED-V lets the value bank read from both content and position latents. The post-softmax gate then allows each token and head to adaptively choose between the content-retrieval route and the position-biased local route. These components raise the score from 56.50 to 57.62.

μMix curriculum. Table 2(c) studies the data curriculum while keeping the DMLA architecture fixed. The standard cosine baseline obtains 57.62. Replacing it with a WSD-style single-stage recipe improves the score to 58.34, indicating that a stable training phase followed by a controlled cooldown is already beneficial for micro-models. Introducing a two-stage curriculum further improves performance to 58.91 by separating general language learning from later commonsense injection. The proposed three-stage curriculum achieves the best score, 59.25, by additionally separating the early syntax/fluency warmup from the language–logic trunk. A four-stage variant slightly drops to 59.18, suggesting that further subdividing the early curriculum introduces unnecessary distribution switches and yields diminishing returns. These results support the design of μMix: tiny models benefit not only from carefully selected data, but also from when different data families are presented during training. Detailed mixture ratios and training-stage definitions are deferred to Appendix C.

6.3 μAgent Analysis

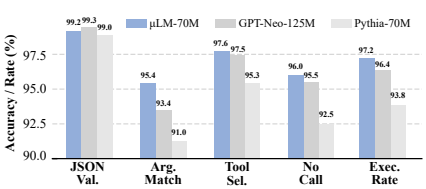


Figure 5: Analysis of μAgent.

The demand for on-device agents on IoT devices has surged rapidly [5]. Agents equipped with basic tool-calling and logical routing capabilities can endow devices (such as adaptive noise cancellation in hearing aids or autonomous obstacle avoidance in unmanned equipment) with autonomous decision-making in offline environments [59]. However, strict code-level JSON output, accurate semantic routing, and reliable rejection of inappropriate tool invocations were previously considered advanced capabilities exclusive to LLMs with billions of parameters. To verify the feasibility of using extremely tiny models as a *domain-specific tool-calling router*, we fine-tune μLM-70M into μAgent. The fine-tuning corpus contains approximately 320M tokens curated across 8 high-frequency on-device scenarios such as weather queries and device control, covering multi-tool selection, argument slot-filling, and negative-sample defense. We compare

μAgent against two same-scale baselines, Pythia-70M and GPT-Neo-125M, all fine-tuned under an identical recipe and evaluated on a shared 1,000-example out-of-distribution benchmark that injects unseen multinational city entities, redundant colloquial prefixes, and adversarial hard negatives. Detailed training and evaluation protocols are deferred to Appendix F.

As shown in Figure 5, μAgent matches or outperforms both baselines across five core metrics. It exhibits remarkable robustness on fundamental JSON validity (JSON Val., 99.2%), zero-shot generalization on out-of-vocabulary noisy entities (Arg. Match, 95.4%, +2.0 over GPT-Neo-125M and +4.4 over Pythia-70M), tool selection (Tool Sel., 97.6%), reliable rejection of chitchat (No Call, 96.0%), and end-to-end deployable behavior (Exec. Rate, 97.2%, +0.8 and +3.4 respectively). The consistent advantage over Pythia-70M, an exactly parameter-matched baseline, confirms that the gain stems from μLM's pre-training rather than tuning effort, and that extremely tiny models are fully capable of structured intent-parsing tasks. This provides promising technical validation for deploying fully local, privacy-preserving agents on ubiquitous yet resource-constrained edge devices.

6.4 On-Device Profiling

To verify the deployment feasibility of μLM on ultra-constrained edge hardware, we profile three variants directly on a Raspberry Pi Zero, one of the most resource-restricted development boards, with a CPU clocked below 1.0 GHz and less than 500 MB of system memory. We deliberately disable any third-party operator accelerators, GEMM tuning libraries, or hardware-specific kernels: every result reported is from pure CPU forward execution, reflecting the model's native compute and memory footprint. Table 3 reports the on-board BF16 model size and the runtime peak RAM during decode for generation lengths $S \in \{1, 32, 64, 128\}$. The peak RAM is measured as the memory footprint of maximum activation tensors plus the KV cache. Thanks to the latent-cache compression in DMLA, the KV-cache memory scaling remains minimal even at $S = 128$. We also report the TTFT/TBT using a 10-token input prompt. Latency results show that all three variants maintain a TBT well below 200 ms with throughputs up to 10.4 tokens/s.

Table 3: On-device profiling of μLM on RPi Zero.

Model	Size (MB)	Runtime RAM (MB)				Latency TTFT/TBT
		S=1	32	64	128	
μLM-70M	148.3	0.04	1.18	2.36	4.72	990/186 ms
μLM-50M	110.2	0.04	1.21	2.43	4.85	830/142 ms
μLM-30M	74.6	0.04	1.16	2.33	4.65	526/96 ms

7 Discussion

μLM makes several deliberate trade-offs for the sub-100M on-device regime. First, given the extremely limited parameter budget, the model is not intended to store dense world knowledge or support broad code/math synthesis. Instead, μLM prioritizes fluent language understanding, commonsense reasoning, and reliable instruction following, which are more relevant to ultra-constrained deployment. Extending it to specialized domains such as mathematics or code remains an important future direction, potentially following specialist-model paradigms such as Qwen2.5-Math [60] and Qwen2.5-Coder [61]. Second, DMLA is designed for the memory-first setting where aggressive latent compression makes every head dimension valuable. As model size and latent width increase, a monolithic latent may be sufficient to encode both semantic and positional information, so DMLA should be viewed as a sub-100M architecture rather than a universal replacement for MLA. Finally, μLM currently targets English only because multilingual vocabularies would make embedding tables dominate the parameter budget. Future work may explore vocabulary pruning or language-aware vocabulary sharing to support multilingual μLM under similarly tight constraints. We see language models for ubiquitous yet constrained devices as an emerging frontier where standard scaling intuitions may break down, inviting fundamental rethinking of architecture and data design.

8 Conclusion

We rethink sub-100M SLM design, where the binding constraint is total memory footprint rather than raw compute, and propose μLM, coupling a memory-first architecture (DMLA) with a capacity-aware multi-stage curriculum (μMix). At 70M parameters, μLM surpasses all sub-300M baselines and rivals several sub-1B models; even at 9M, it remains competitive across baselines, demonstrating that capability density at this scale is unlocked by deliberate co-design. Fine-tuned into a tool-calling agent, μLM outperforms same-scale baselines and delivers a 97.2% end-to-end executable rate.

References

[1] Chaojun Xiao, Jie Cai, Weilin Zhao, Biyuan Lin, Guoyang Zeng, Jie Zhou, Zhi Zheng, Xu Han, Zhiyuan Liu, and Maosong Sun. Densing law of llms. *Nature Machine Intelligence*, pages 1–11, 2025.

[2] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.

[3] MiniCPM Team, Chaojun Xiao, Yuxuan Li, Xu Han, Yuzhuo Bai, Jie Cai, Haotian Chen, Wentong Chen, Xin Cong, Ganqu Cui, et al. Minicpm4: Ultra-efficient llms on end devices. *arXiv preprint arXiv:2506.07900*, 2025.

[4] Vasileios Tsoukas, Anargyros Gkogkidis, Eleni Boumpa, and Athanasios Kakarountas. A review on the emerging technology of tinyml. *ACM Comput. Surv.*, 56(10), June 2024. ISSN 0360-0300. doi: 10.1145/3661820. URL `https://doi.org/10.1145/3661820`.

[5] Jiajun Xu, Zhiyuan Li, Wei Chen, Qun Wang, Xin Gao, Qi Cai, and Ziyuan Ling. On-device language models: A comprehensive review. *arXiv preprint arXiv:2409.00088*, 2024.

[6] Raspberry Pi. Rp3a0. `https://www.raspberrypi.com/documentation/computers/processors.html#rp3a0`, 2026. Accessed 13-Apr-2026.

[7] ST. Stm32n657. `https://www.st.com/en/microcontrollers-microprocessors/stm32n657x0.html`, 2026. Accessed 13-Apr-2026.

[8] Milk-V. Cv1800b. `https://milkv.io/chips/cv1800b`, 2026. Accessed 13-Apr-2026.

[9] Josh Millar, Yushan Huang, Sarab Sethi, Hamed Haddadi, and Anil Madhavapeddy. Benchmarking ultra-low-power μnpus. In *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking*, ACM MOBICOM '25, page 1060–1074, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400711299. doi: 10.1145/3680207.3765264.

[10] Rong Ding, Haiming Jin, Ningzhi Zhu, Zijie Chen, Yi Fu, Fengyuan Zhu, Guiyun Fan, Xiaohua Tian, and Linghe Kong. Bluetooth-enabled transparent rf sensing. In *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking*, pages 186–200, 2025.

[11] Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International conference on machine learning*, pages 2397–2430. PMLR, 2023.

[12] Nolan Dey, Gurpreet Gosal, Hemant Khachane, William Marshall, Ribhu Pathria, Marvin Tom, Joel Hestness, et al. Cerebras-gpt: Open compute-optimal language models trained on the cerebras wafer-scale cluster. *arXiv preprint arXiv:2304.03208*, 2023.

[13] Dan John Velasco and Matthew Theodore Roque. Rethinking the role of text complexity in language model pretraining. In *Proceedings of the First BabyLM Workshop*, pages 1–28, 2025.

[14] Ronen Eldan and Yuanzhi Li. Tinystories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023.

[15] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W Mahoney, and Kurt Keutzer. Ai and memory wall. *IEEE Micro*, 44(3):33–39, 2024.

[16] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[17] Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, 2023.

[18] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[19] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, DDL Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 10, 2022.

[20] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, et al. Smollm2: When smol goes big–data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*, 2025.

[21] Lintang Sutawika, Hailey Schoelkopf, Leo Gao, Baber Abbasi, Stella Biderman, Jonathan Tow, Ben Fattori, Charles Lovering, Jason Phang, and Anish Thite. Eleutherai/lm-evaluation-harness, January 2026. URL `https://doi.org/10.5281/zenodo.18394108`.

[22] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *NeurIPS*, 2023.

[23] Long Cheng, Yan Gu, Qingzhi Liu, Lei Yang, Cheng Liu, and Ying Wang. Advancements in accelerating deep neural network inference on aiot devices: A survey. *IEEE Transactions on Sustainable Computing*, 9 (6):830–847, 2024. doi: 10.1109/TSUSC.2024.3353176.

[24] Zijie Chen, Yiming Gao, and Junrui Liang. Lopdm: A low-power on-device predictive maintenance system based on self-powered sensing and tinyml. *IEEE Transactions on Instrumentation and Measurement*, 72: 1–13, 2023.

[25] Ji Lin, Wei-Ming Chen, Yujun Lin, Chuang Gan, Song Han, et al. Mcunet: Tiny deep learning on iot devices. *Advances in neural information processing systems*, 33:11711–11722, 2020.

[26] Ji Lin, Ligeng Zhu, Wei-Ming Chen, Wei-Chen Wang, and Song Han. Tiny machine learning: Progress and futures [feature]. *IEEE Circuits and Systems Magazine*, 23(3):8–34, 2023.

[27] Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, et al. Mobilellm: Optimizing sub-billion parameter language models for on-device use cases. In *Forty-first International Conference on Machine Learning*, 2024.

[28] Omkar Thawakar, Ashmal Vayani, Salman Khan, Hisham Cholakkal, Rao Muhammad Anwer, Michael Felsberg, Timothy Baldwin, Eric P. Xing, and Fahad Shahbaz Khan. Mobillama: Towards accurate and lightweight fully transparent gpt, 2024.

[29] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, et al. Qwen2.5 technical report, 2025. URL `https://arxiv.org/abs/2412.15115`.

[30] Fanxu Meng, Pingzhi Tang, Xiaojuan Tang, Zengwei Yao, Xing Sun, and Muhan Zhang. Transmla: Multi-head latent attention is all you need. *arXiv preprint arXiv:2502.07864*, 2025.

[31] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[32] Ofir Press, Noah Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations*, 2022. URL `https://openreview.net/forum?id=R8sQPpGCv0`.

[33] Niklas Muennighoff, Alexander Rush, Boaz Barak, Teven Le Scao, Nouamane Tazi, Aleksandra Piktus, Sampo Pyysalo, Thomas Wolf, and Colin A Raffel. Scaling data-constrained language models. *Advances in Neural Information Processing Systems*, 36:50358–50376, 2023.

[34] Guilherme Penedo, Hynek Kydlíček, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, Thomas Wolf, et al. The fineweb datasets: Decanting the web for the finest text data at scale. *Advances in Neural Information Processing Systems*, 37:30811–30849, 2024.

[35] Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, Sedrick Keh, Kushal Arora, et al. Datacomp-lm: In search of the next generation of training sets for language models. *Advances in Neural Information Processing Systems*, 37:14200–14282, 2024.

[36] Loubna Ben Allal, Anton Lozhkov, and Daniel van Strien. Cosmopedia: how to create large-scale synthetic data for pre-training large language models. Hugging Face Blog, 2024.

[37] Marah Abdin, Jyoti Aneja, et al. Phi-3 technical report: A highly capable language model locally on your phone, 2024. URL `https://arxiv.org/abs/2404.14219`.

[38] Utkarsh Saxena, Gobinda Saha, Sakshi Choudhary, and Kaushik Roy. Eigen attention: Attention in low-rank space for kv cache compression. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 15332–15344, 2024.

[39] Prajwal Singhania, Siddharth Singh, Shwai He, Soheil Feizi, and Abhinav Bhatele. Loki: Low-rank keys for efficient sparse attention. *Advances in Neural Information Processing Systems*, 37:16692–16723, 2024.

[40] Seijin Kobayashi, Yassir Akram, and Johannes Von Oswald. Weight decay induces low-rank attention layers. *Advances in Neural Information Processing Systems*, 37:4481–4510, 2024.

[41] Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pages 5797–5808, 2019.

[42] Pengcheng He, Jianfeng Gao, and Weizhu Chen. Debertav3: Improving deberta using electra-style pre-training with gradient-disentangled embedding sharing. *arXiv preprint arXiv:2111.09543*, 2021.

[43] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[44] Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in neural information processing systems*, 32, 2019.

[45] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[46] Zhenting Qi, Fan Nie, Alexandre Alahi, James Zou, Himabindu Lakkaraju, Yilun Du, Eric Xing, Sham Kakade, and Hanlin Zhang. Evolm: In search of lost language model training dynamics. *arXiv preprint arXiv:2506.16029*, 2025.

[47] Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, et al. Textbooks are all you need. *arXiv preprint arXiv:2306.11644*, 2023.

[48] Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, et al. Minicpm: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*, 2024.

[49] Alexander Hägele, Elie Bakouch, Atli Kosson, Loubna B Allal, Leandro Von Werra, and Martin Jaggi. Scaling laws and compute-optimal training beyond fixed training durations. *Advances in Neural Information Processing Systems*, 37:76232–76264, 2024.

[50] Mahnaz Koupaee and William Yang Wang. Wikihow: A large scale text summarization dataset. *arXiv preprint arXiv:1810.09305*, 2018.

[51] Alex Warstadt, Alicia Parrish, Haokun Liu, Anhad Mohananey, Wei Peng, Sheng-Fu Wang, and Samuel R Bowman. Blimp: The benchmark of linguistic minimal pairs for english. *Transactions of the Association for Computational Linguistics*, 8:377–392, 2020.

[52] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. Boolq: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 conference of the north American chapter of the association for computational linguistics: Human language technologies, volume 1 (long and short papers)*, pages 2924–2936, 2019.

[53] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.

[54] Maarten Sap, Hannah Rashkin, Derek Chen, Ronan Le Bras, and Yejin Choi. Social iqa: Commonsense reasoning about social interactions. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 4463–4473, 2019.

[55] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439, 2020.

[56] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pages 4791–4800, 2019.

[57] Gabriel Haas, Michael Rietzler, Matt Jones, and Enrico Rukzio. Keep it short: A comparison of voice assistants' response behavior. In *Proceedings of the 2022 CHI conference on human factors in computing systems*, pages 1–12, 2022.

[58] Tawfiq Ammari, Jofish Kaye, Janice Y Tsai, and Frank Bentley. Music, search, and iot: How people (really) use voice assistants. *ACM Transactions on Computer-Human Interaction (TOCHI)*, 26(3):1–28, 2019.

[59] Peter Udo Diehl, Yosef Singer, Hannes Zilly, Uwe Schönfeld, Paul Meyer-Rachner, Mark Berry, Henning Sprekeler, Elias Sprengel, Annett Pudszuhn, and Veit M Hofmann. Restoring speech intelligibility for hearing aid users with deep learning. *Scientific Reports*, 13(1):2719, 2023.

[60] An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2. 5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.

[61] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

[62] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 1715–1725, 2016.

[63] Phillip Rust, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. How good is your tokenizer? on the monolingual performance of multilingual language models. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 3118–3135, 2021.

[64] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[65] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[66] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in neural information processing systems*, 35:30318–30332, 2022.